

Kyrozen 分阶段开发实施方案

总体开发原则

Kyrozen 的开发不能按照“先创建十几个 Agent 文件，再慢慢填功能”的方式进行。每个阶段都必须形成一个能够实际运行和验证的成果，并为下一阶段提供稳定基础。

整个开发顺序遵循：

底层 Agent 能力

- 项目工作空间
- 问题发现
- 市场研究
- 产品规划
- 软件开发
- 硬件开发
- 测试验证
- 自学习
- 完整产品化

每个阶段完成后都应：

- 能够独立运行；
- 有明确的输入和输出；
- 保存真实项目数据；
- 包含自动测试；
- 不依赖尚未开发完成的后续模块；
- 不用虚假或硬编码结果冒充功能完成。

阶段一：建立 Kyrozen Core

阶段目标

将现有 OpenKyrozen 转化为 Kyrozen 可以长期使用的稳定 Agent 基础。

这一阶段不开发市场调研、产品规划或者硬件功能，而是先保证 Kyrozen 具备可靠的推理、工具调用、任务执行、记忆和模型管理能力。

OpenKyrozen 当前已经拥有多模型、文件操作、终端执行、Git、网络搜索、长期记忆、插件和自学习等能力。这个阶段的重点不是推倒重写，而是整理和升级这些能力，使其能够支撑后续完整产品开发流程。

主要实现内容

Kyrozen Core 需要统一管理：

- OpenAI、Claude、Gemini、DeepSeek、Ollama 等模型；
- 不同任务使用不同模型的选择规则；
- 工具注册、调用和结果返回；
- 文件与终端操作；
- Git 仓库操作；
- 网络搜索和网页读取；
- 长任务执行和进度记录；
- 错误处理与重试；
- 用户批准和高风险操作确认；
- 基础长期记忆；
- Agent 运行日志和成本记录。

现有 OpenKyrozen 中集中在单个核心文件内的功能，应整理成职责明确的模块。Kyrozen 后续所有专业 Agent 都通过同一套 Core 调用模型和工具，而不是各自重新实现一遍。

工具参数应逐渐从模糊的字符串格式升级为结构化参数，使系统能够检查参数是否完整，并保存每次工具调用的真实记录。

阶段交付成果

完成后应得到：

- 一个独立可运行的 Kyrozen Core；
- 统一的模型接口；
- 统一的工具执行接口；
- 基础任务管理器；
- 基础记忆接口；
- 权限和确认机制；
- 完整运行日志；
- 基础自动测试；
- 从 OpenKyrozen 迁移而来的通用能力。

阶段完成标准

用户可以向 Kyrozen 提交一个复杂任务，Kyrozen能够：

1. 理解任务；
2. 生成执行计划；
3. 调用多个工具；
4. 保存进度；
5. 在失败后重试或给出真实错误；

6. 最终返回执行结果。

此时它仍然是通用 Agent，但底层已经足够稳定，可以承载产品开发功能。

阶段二：建立项目工作空间与产品开发流程

阶段目标

让 Kyrozen 从“处理一次聊天任务的 Agent”变成“长期管理完整项目的 Agent”。

Kyrozen 的核心单位不是一次对话，而是一个项目。每个项目都要有独立状态、资料、决策、文件、记忆和任务。

主要实现内容

用户能够创建、打开、暂停、继续和归档项目。

每个项目至少保存：

- 项目名称；
- 原始想法或问题；
- 目标用户；
- 当前开发阶段；
- 软件、硬件、混合或未确定路线；
- 已完成任务；
- 下一步任务；
- 调研资料；
- 产品文档；
- 设计决策；
- 项目文件；
- 软件代码；
- 硬件方案；
- BOM；
- 测试结果；
- 用户反馈；
- 风险；
- 历史版本。

系统需要建立产品生命周期状态，使 Kyrozen 知道项目当前是在问题发现、市场研究、产品定义、设计开发还是测试验证，而不是依靠模型从聊天记录中猜测。

所有重要决策都需要保存原因。例如：

选择 ESP32-S3，而不是普通 ESP32，因为项目需要 USB 和更多内存。

当用户以后重新打开项目时，Kyrozen要能够解释之前为什么这样选择。

系统还要支持项目产物，包括 Problem Brief、市场报告、PRD、设计方案、BOM和测试报告。这些产物需要能够更新和保存版本，而不是每次重新生成一份互相矛盾的内容。

阶段交付成果

完成后应得到：

- 项目创建和管理系统；
- 项目生命周期状态；
- 项目任务与进度系统；
- 项目文件工作区；
- 项目产物管理；
- 决策记录；
- 项目级记忆；
- 项目恢复能力；
- 基础项目主页。

阶段完成标准

用户创建一个项目并完成几次对话后，即使重启 Kyrozen，也仍然能够看到：

- 项目要解决的问题；
- 当前做到哪一步；
- 已经完成什么；
- 做过哪些决定；
- 当前风险；
- 下一步应该做什么。

在这一阶段之后，所有后续功能都必须围绕项目运行，不能再只依赖全局聊天记录。

阶段三：实现问题发现与问题定义

阶段目标

让 Kyrozen 从用户的模糊想法、经历或困扰中，找到真正需要解决的问题。

这是 Kyrozen 与普通编程 Agent 的关键区别。普通 Agent 通常要求用户已经知道要开发什么，而 Kyrozen 可以从：

“我跑步时经常觉得音乐不合适。”

这种模糊描述开始工作。

主要实现内容

Kyrozen需要通过自然对话理解：

- 谁遇到问题；
- 问题发生在什么场景；
- 问题出现频率；
- 问题造成什么影响；
- 用户当前如何解决；
- 现有方式为什么不够好；
- 用户真正希望获得什么结果；
- 用户最初提出的产品形式是否只是一个未经验证的假设。

用户可以通过文字、图片、视频、已有产品或工作流程来描述问题。

Kyrozen不应一次询问大量专业问题，而要根据已有信息逐步提出少量、容易回答的问题。对完全没有产品经验的用户，系统应使用普通语言，不要求用户理解“需求分析”“价值主张”等术语。

这一阶段还需要允许 Kyrozen判断：

- 这是一个真实问题；
- 只是个人偏好；
- 问题描述仍然不清楚；
- 已经有明显的现成解决方案；
- 可能不需要开发产品。

最终生成结构化的 Problem Brief。

阶段交付成果

Problem Brief 至少包括：

- 目标用户；
- 使用场景；
- 表面问题；
- 深层需求；
- 当前解决方式；
- 当前方案的不足；
- 问题频率和影响；
- 尚未验证的假设；
- 初步产品机会；
- 建议继续调查的问题。

同时保存形成这些结论的对话和证据。

阶段完成标准

给 Kyrozen 多种不同输入，例如：

- 一个完整产品想法；
- 一句模糊困扰；
- 某类用户的问题；
- 一个已有产品的缺点；
- 一个可能根本不需要开发产品的问题。

Kyrozen 都能够生成合理的 Problem Brief，并且不会直接跳到写代码或推荐硬件。

阶段四：实现市场调研与产品机会判断

阶段目标

让 Kyrozen 判断问题是否值得解决，以及市场上是否已经存在足够好的解决方案。

这一阶段必须以真实网络资料为依据，不能依赖模型记忆生成看似合理的竞品名称或市场数据。

主要实现内容

Kyrozen 根据 Problem Brief 自动制定调研方向，搜索和分析：

- 已有消费产品；
- 软件和 App；
- 公司与创业项目；
- GitHub 开源项目；
- Arduino 和 DIY 项目；
- 众筹项目；
- 专利；
- 学术论文；
- 用户评价；
- 论坛和社区讨论；
- 已停止销售或失败的产品；
- 不开发新产品的替代解决方法。

每条重要信息都要保存：

- 来源；
- 标题；
- 链接； -发布日期；

- 搜索日期；
- 内容摘要；
- 支持的结论；
- 可信度；
- 是否属于推断。

Kyrozen需要分析的不只是“有没有类似产品”，还包括：

- 竞品解决了哪些问题；
- 主要功能；
- 目标用户；
- 定价；
- 用户喜欢什么；
- 用户抱怨什么；
- 为什么产品成功或失败；
- 现有方案是否已经足够；
- 新项目可能有什么差异；
- 是否存在明显专利、法规或安全风险。

系统最后要做出诚实判断。可能的建议包括继续、缩小范围、改变目标用户、改用软件、改用硬件、使用现成产品、暂停或放弃。

阶段交付成果

完成后生成 Market Research Report，包括：

- 问题是否真实存在；
- 主要竞品；
- 竞品对比；
- 开源项目；
- 用户反馈；
- 替代方案；
- 技术路线；
- 市场缺口；
- 机会和风险；
- 继续开发建议；
- 全部资料来源。

阶段完成标准

对一个已经存在大量竞品的想法，Kyrozen不能错误地声称“市场空白”。

对一个缺乏证据的想法，Kyrozen必须标记证据不足。

对一个明显已有简单解决方法的问题，Kyrozen应能建议用户采用现有方案，而不是强行创建新项目。

阶段五：实现产品规划与方案决策

阶段目标

将经过验证的问题转化为明确、可执行、可测试的产品方案。

这一阶段由 Kyrozen 承担产品经理和方案规划团队的职责，决定应该做什么，而不是立即进入开发。

主要实现内容

Kyrozen 需要根据问题和市场研究结果建立：

- 产品目标；
- 核心用户；
- 价值主张；
- 用户使用流程；
- 用户故事；
- 核心功能；
- 功能优先级；
- 性能要求；
- 成本限制；
- 时间限制；
- 隐私和安全需求；
- 成功标准；
- 第一版明确不做的内容。

Kyrozen 需要帮助用户缩小 MVP。对于范围过大的想法，系统必须主动删除与核心验证无关的功能，而不是接受全部需求。

系统还要提出多个不同解决方案，例如：

- 纯软件方案；
- 纯硬件方案；
- 软硬件混合方案；
- 使用现成产品组合；
- 最低成本方案；
- 最容易实现方案；
- 最佳体验方案。

每种方案需要比较：

- 能否解决问题；
- 成本；
- 难度；

- 开发时间；
- 使用门槛；
- 稳定性；
- 扩展性；
- 风险。

系统不能自行替用户完成所有重大产品决策，而应向用户展示推荐和理由，让用户确认。

阶段交付成果

完成后生成：

- Product Brief；
- PRD；
- MVP 范围；
- 用户流程；
- 功能优先级；
- 成功指标；
- 软件、硬件、混合或非开发路线决定；
- 方案对比；
- 最终方案；
- 决策记录；
- 初步执行计划。

阶段完成标准

用户能够清楚回答：

- Kyrozen准备做什么；
- 为谁做；
- 解决什么问题；
- 第一版包含什么；
- 第一版不包含什么；
- 怎么判断第一版成功；
- 为什么选择当前方案；
- 为什么不选择其他方案。

完成这一阶段后，项目才可以进入实际开发。

阶段六：实现软件产品开发路线

阶段目标

让 Kyrozen能够根据已经批准的产品方案，完成一个真实可运行的软件原型。

第一版重点支持网站、Web App、简单 SaaS、AI 工具、自动化工具和简单桌面应用，不追求立即覆盖所有软件平台。

主要实现内容

Kyrozen根据 PRD和用户约束完成：

- 用户体验和页面流程；
- 界面结构；
- 技术选型；
- 软件架构；
- 数据模型；
- API设计；
- 项目目录；
- 代码开发；
- Git管理；
- 依赖安装；
- 本地运行；
- 自动测试；
- 错误修复；
- 基础部署；
- 使用说明。

开发过程应直接复用 Kyrozen Core 的文件、终端和 Git 能力，并把所有代码保存在项目工作区。

Kyrozen需要保持产品规划和代码开发一致。例如，PRD中没有包含的复杂功能，不应被编程 Agent擅自加入。产品需求发生变化时，需要先更新产品决策，再修改代码。

系统应能够调用专门的编程 Agent或 Codex完成大量实际编码，但 Kyrozen负责：

- 提供完整上下文；
- 明确任务；
- 检查结果；
- 运行测试；
- 记录变更；
- 判断是否符合产品目标。

阶段交付成果

完成后应得到：

- 可运行的软件项目；
- 完整源代码；
- 基础 UI；
- 数据存储；

- 核心功能；
- 自动测试；
- README；
- Git提交历史；
- 运行或部署方式；
- 与PRD对应的功能完成记录。

阶段完成标准

至少选择多个参考项目进行验证，例如：

- 简单 Web 工具；
- 带数据库的 Web App；
- AI功能应用；
- 自动化工具。

用户仅通过 Kyrozen 的项目流程，就能够从批准的 PRD 得到可运行的软件原型，而不需要自己手动建立代码项目。

阶段七：实现硬件与软硬件混合开发路线

阶段目标

让 Kyrozen能够把一个硬件或混合产品方案落实成具体可购买、可连接、可编译、可组装的真实原型。

第一版重点支持 Arduino、ESP32、Raspberry Pi及常见低压传感器和模块。

主要实现内容

Kyrozen根据产品需求完成：

- 硬件系统架构；
- 主控制器选择；
- 传感器和输出设备选择；
- 通信方式；
- 电源需求；
- 引脚分配；
- 电路连接；
- 固件架构；
- 软件与硬件接口；
- 完整BOM；
- 具体型号；
- 价格和购买链接；

- 替代型号；
- 组装步骤；
- 固件生成；
- 编译；
- 烧录；
- 串口日志分析；
- 基础硬件调试。

采购结果不能只写“ESP32”或“MPU6050”，而应尽量明确到开发板、模块和版本，并说明为什么选择。

系统需要检查：

- 供电电压；
- 逻辑电平；
- 电流；
- GPIO冲突；
- I²C地址；
- 通信接口；
- 电源能力；
- 元件兼容性；
- 用户是否需要焊接；
- 是否遗漏连接线、适配器或保护元件。

软硬件混合项目需要同时管理设备固件、网页或App、云端服务和通信协议，确保各部分使用一致的数据格式。

还需要实现基础 Local Hardware Bridge，使 Kyrozen可以在用户电脑上调用 Arduino CLI或 PlatformIO，识别串口、编译、烧录和读取日志。

阶段交付成果

完成后应得到：

- 硬件系统方案；
- 具体BOM；
- 可用购买链接；
- 接线图；
- 引脚表；
- 电源说明；
- 固件源码；
- 可重复的编译配置；
- 烧录工具；
- 串口监视；
- 分步骤制作指导；
- 硬件测试程序；
- 混合项目的软件部分。

阶段完成标准

使用多个真实参考项目进行验证。用户应该能够：

1. 获得正确的采购清单；
2. 买到具体元件；
3. 根据指导完成连接；
4. 让Kyrozen编译并烧录程序；
5. 读取设备输出；
6. 在出现错误时获得有效诊断；
7. 最终让设备完成核心功能。

不能只以“代码能够编译”作为完成标准，至少需要真实硬件验证。

阶段八：实现测试、产品验证与迭代闭环

阶段目标

让 Kyrozen判断产品是否真正解决了最初的问题，而不是把“程序运行”误认为“产品完成”。

这一阶段同时服务软件、硬件和混合项目。

主要实现内容

Kyrozen需要将 PRD中的每条核心需求转换为可执行测试。

软件项目可包括：

- 功能测试；
- 界面测试；
- API测试；
- 性能测试；
- 安全测试；
- 多设备测试；
- 可用性测试。

硬件项目可包括：

- 编译测试；
- 单元模块测试；
- 集成测试；
- 传感器精度；
- 功耗；

- 续航；
- 延迟；
- 稳定性；
- 长时间运行；
- 异常状态。

当测试失败时，Kyrozen应采用证据驱动的调试过程：

观察结果
→ 比较预期
→ 提出可能原因
→ 设计最小验证实验
→ 执行实验
→ 排除原因
→ 修改产品
→ 重新测试

Kyrozen还需要帮助用户进行真实用户验证，例如：

- 用户访谈；
- 产品试用；
- 反馈问卷；
- 使用观察；
- 对比测试；
- 核心指标记录。

测试和反馈不能只保存在聊天里，而要形成测试记录，并关联到具体需求和产品版本。

阶段交付成果

完成后生成：

- Test Plan；
- 测试用例；
- 自动测试结果；
- 硬件测试记录；
- 错误日志；
- 调试报告；
- 用户反馈；
- 产品验证报告；
- 未解决问题；
- 下一轮迭代列表。

阶段完成标准

Kyrozen能够明确回答：

- 哪些需求已经通过测试；
- 哪些需求失败；
- 为什么失败；
- 产品是否改善了最初问题；
- 哪些功能应该保留；
- 哪些功能应该修改；
- 是否值得进入下一轮开发。

完成后，用户可以从验证结果直接启动新的产品迭代，而不是重新创建项目。

阶段九：实现项目自学习与主动改进

阶段目标

将 OpenKyrozen 原有的自学习能力转化为专门服务产品开发的持续学习系统。

Kyrozen不仅记住用户说过什么，还应从项目成功、失败、决策和测试中形成可复用经验，并在空闲时主动寻找项目改进机会。

主要实现内容

Kyrozen需要建立不同层级的学习内容：

- 用户偏好；
- 用户技术能力；
- 项目事实；
- 产品决策；
- 已验证成功方案；
- 已验证失败方案；
- 软件开发经验；
- 硬件兼容经验；
- 采购经验；
- 测试经验；
- 跨项目可复用知识。

记忆必须区分：

- 未验证推测；
- 用户提供信息；

- 外部资料；
- 单次测试结果；
- 多次验证经验；
- 可跨项目复用知识。

空闲时，Kyrozen可以分析：

- 项目是否偏离原始问题；
- 是否存在尚未验证的关键假设；
- 是否遗漏测试；
- 软件架构是否过度复杂；
- 硬件是否存在重复元件；
- BOM是否可以降低成本；
- 是否有新的竞品；
- 是否有新的元件或技术；
- 是否有失败模式反复出现；
- 哪些任务可以自动化。

Kyrozen只能提出改进建议，不能在用户不知情时修改项目。

用户必须能够：

- 查看记忆；
- 删除错误记忆；
- 修改记忆；
- 标记记忆为私有；
- 禁止跨项目使用；
- 关闭空闲分析；
- 接受、忽略或延后建议。

阶段交付成果

完成后应得到：

- 用户记忆；
- 项目记忆；
- 失败知识库；
- 成功经验库；
- 跨项目经验；
- 空闲项目分析；
- 改进建议中心；
- 记忆查看和删除功能；
- 建议接受与反馈机制。

阶段完成标准

Kyrozen在多个项目中能够：

- 记住用户之前的选择和限制；
- 避免重复已经确认的错误；
- 使用已验证经验解决相似问题；
- 主动提出有依据的改进建议；
- 清楚解释建议来自哪些项目事实或测试结果；
- 在用户删除记忆后不再继续使用它。

自学习不能只表现为“存入了更多向量数据”，而必须实际改善后续项目决策。

阶段十：完成正式产品界面、安全系统与Beta发布

阶段目标

将前九个阶段形成的能力整合为普通用户真正能够使用的 Kyrozen 产品。

此前阶段可以主要依赖开发界面和内部工具，但最终产品不能只是聊天框，也不能要求用户理解底层 Agent结构。

主要实现内容

建立完整产品界面，包括：

项目主页

显示：

- 项目目标；
- 当前阶段；
- 当前进度；
- 下一步；
- 主要风险；
- 当前预算；
- 最近产物；
- Kyrozen最新建议。

对话和项目画布

用户通过对话提出问题，右侧或独立页面展示：

- Problem Brief；

- 市场研究；
- PRD；
- 方案设计；
- 软件；
- 硬件；
- 采购；
- 测试；
- 迭代。

决策中心

集中显示需要用户确认的事项，例如：

- 是否继续项目；
- 选择哪个MVP；
- 使用软件还是硬件方案；
- 选择哪个元件；
- 是否执行高风险操作。

采购中心

管理：

- 必买；
- 可选；
- 用户已有；
- 已购买；
- 已到货；
- 替代型号；
- 总成本；
- 商家和链接。

制作模式

面向手机和平板，向零基础用户一次显示一个操作步骤，并支持上传照片、视频、日志和错误。

改进建议中心

显示 Kyrozen在自学习和空闲分析中发现的：

- 风险；
- 市场变化；
- 成本优化；
- 技术优化；
- 测试遗漏；
- 新产品机会。

产品还需要完成：

- 用户账户；
- 多项目；
- 数据权限；
- 项目导出；
- 项目删除；
- 隐私控制；
- 日志审计；
- 高风险工具确认；
- 本地设备权限；
- 错误反馈；
- 使用统计；
- Beta测试机制。

阶段交付成果

完成后应得到：

- 正式 Kyrozen Web 产品；
- 用户账户和多项目；
- 完整项目工作空间；
- 软件和硬件路线；
- 本地硬件桥接；
- 项目导出；
- 自学习控制；
- 权限和隐私设置；
- 错误监控；
- Beta 文档；
- 示例项目；
- 新用户引导。

阶段完成标准

邀请一个完全不了解 Kyrozen、编程和电子开发的测试用户。

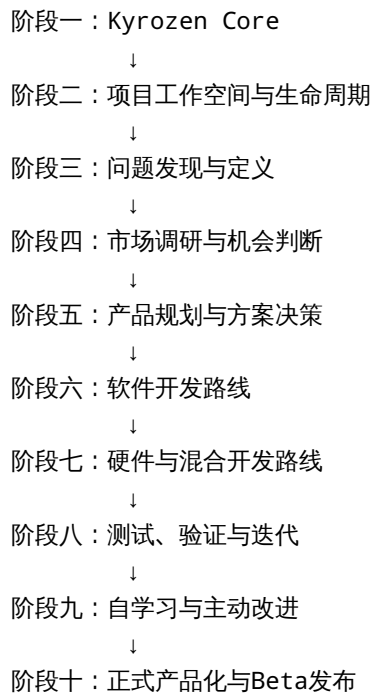
该用户应该能够在不阅读开发文档的情况下：

1. 创建项目；
2. 描述问题；
3. 完成问题定义；
4. 查看市场调研；
5. 确认 MVP；
6. 选择软件或硬件路线；
7. 完成一个真实原型；

8. 执行测试；
9. 查看改进建议；
10. 之后重新打开项目继续开发。

当这一完整流程能够被普通用户稳定完成，Kyrozen才算达到可公开测试的Beta状态。

最终阶段关系



其中阶段六和阶段七在实际开发中可以先后完成，但最终必须共用前五个阶段的产品流程，并共同接入测试、自学习和正式产品界面。

Kyrozen第一版最重要的目标不是“拥有最多Agent”，而是：

一个没有专业背景的用户，能够从一个模糊问题开始，在Kyrozen帮助下完成市场判断、产品规划，并最终做出一个真正能够运行的软件或硬件产品原型。